

Сравнение и анализ оптимизаций компиляторов

Теврюков Илья Андреевич

Группа: 25.Б42

Кафедра: системного программирования

Научный руководитель: Гориховский Вячеслав Игоревич

Номер семестра практики: 2

1. Введение

При разработке программного обеспечения часто возникает необходимость внедрения криптографических библиотек в существующие проекты. Однако выбор компилятора для компиляции такой библиотеки может существенно повлиять на производительность и размер итогового продукта. Различные компиляторы по-разному оптимизируют один и тот же код, что приводит к различиям в скорости выполнения и размере бинарного файла. В данной работе исследуются три компилятора: GCC, AOCC и ICX. Анализ включает изучение генерируемого ассемблерного кода, замеры производительности и размера бинарных файлов при различных уровнях оптимизации: -O0, -O2, -Os. В качестве тестовых сценариев используются тестовая программа `optbench.c` и алгоритм шифрования AES-128.

2. Постановка задачи

Цель работы: провести сравнительный анализ оптимизаций компиляторов GCC, AOCC и ICX, оценить их влияние на производительность и размер кода на программах `aes.c` и `optbench.c`.

Задачи:

1. Выбрать сценарий применения языка C и описать характеристики тестовой системы.
2. Обосновать выбор компиляторов для исследования.
3. Провести замеры времени исполнения и размера исполняемых файлов для каждого компилятора при трех уровнях оптимизации: -O0, -O2, -Os.
4. Выполнить анализ ассемблерного кода для каждого компилятора, скомпилированного с флагами -O0 и -O2.
5. Провести сравнительный анализ оптимизации разных компиляторов.
6. Сформулировать выводы о примененных оптимизациях каждого компилятора.

3. Обзор литературы и обоснование выбора компиляторов

Алгоритм AES является симметричным блочным шифром, принятым в качестве стандарта правительством США в 2001 году [1]. В работе используется реализация алгоритма на языке C, доступная в открытом репозитории `tiny-AES` [2], которая распространяется под лицензией Unlicense. Данная реализация выбрана благодаря своей минималистичности и отсутствию аппаратно-зависимых оптимизаций, что позволяет объективно оценить возможности компиляторов.

Для исследования выбраны три компилятора, каждый из которых представляет свой подход:

- **GCC 13.3.0** — выбран как основной компилятор с открытым исходным кодом. Он обеспечивает высокую совместимость и предсказуемый результат, служа основой для сравнения.
- **AOCC 5.0.0** (на базе Clang 17) — компилятор от компании AMD. Он специально оптимизирован под микроархитектуру процессоров AMD Zen. Поскольку тестирование проводится на процессоре Ryzen, ожидается, что он покажет хорошую адаптацию к оборудованию.
- **ICX 2026.0.0** (Intel oneAPI) — компилятор от Intel. Его включение позволяет проверить, насколько эффективно код, сгенерированный ICX, будет работать на процессоре AMD, и какие уникальные оптимизации он применит.

4. Описание предлагаемого решения

Исследование проводилось на системе: ОС Ubuntu 24.04.3 LTS, процессор AMD Ryzen 7 5700U, архитектура x86_64.

Для получения исполняемых файлов и проведения замеров использовались три уровня оптимизации (-O0, -O2, -Os). Для получения ассемблерного кода использовались только флаги -O0 и -O2.

Команды компиляции для алгоритма AES:

- *Генерация ассемблерного кода (флаги -O0 и -O2):*
 - **GCC:** `gcc -O0 -S source/aes/aes.c -o assembler/aes/aes_gcc_O0.s` (и аналогично для -O2)
 - **AOCC:** `clang-17 -O0 -S source/aes/aes.c -o assembler/aes/aes_aocc_O0.s` (и аналогично для -O2)
 - **ICX:** `icx -O0 -S source/aes/aes.c -o assembler/aes/aes_icx_O0.s` (и аналогично для -O2)
- *Сборка исполняемых файлов (флаги -O0, -O2, -Os):*

- **GCC:** gcc -O0 source/aes/aes.c source/aes/test.c -o build/aes_gcc_O0 (и аналогично для -O2, -Os)
- **AOCC:** clang-17 -O0 source/aes/aes.c source/aes/test.c -o build/aes_aocc_O0 (и аналогично для -O2, -Os)
- **ICX:** icx -O0 source/aes/aes.c source/aes/test.c -o build/aes_icx_O0 (и аналогично для -O2, -Os)

Команды компиляции для тестового набора Optbench:

- *Генерация ассемблерного кода (флаги -O0 и -O2):*
 - **GCC:** gcc -O0 -S source/optbench/optbench.c -o assembler/optbench/optbench_gcc_O0.s (и аналогично для -O2)
 - **AOCC:** clang-17 -O0 -S source/optbench/optbench.c -o assembler/optbench/optbench_aocc_O0.s (и аналогично для -O2)
 - **ICX:** icx -O0 -S source/optbench/optbench.c -o assembler/optbench/optbench_icx_O0.s (и аналогично для -O2)
- *Сборка исполняемых файлов (флаги -O0, -O2, -Os):*
 - **GCC:** gcc -O0 source/optbench/optbench.c -o build/optbench_gcc_O0 (и аналогично для -O2, -Os)
 - **AOCC:** clang-17 -O0 source/optbench/optbench.c -o build/optbench_aocc_O0 (и аналогично для -O2, -Os)
 - **ICX:** icx -O0 source/optbench/optbench.c -o build/optbench_icx_O0 (и аналогично для -O2, -Os)

5. Описание экспериментов

Замеры проводились с помощью утилит stat -c "%s" (для определения размера в байтах) и /usr/bin/time (для замера времени, 1000 запусков для статистической значимости). Полные таблицы результатов размещены в репозитории [8].

Анализ замеров времени исполнения и объёма исполняемого файла:

- **Optbench:** AOCC генерирует наиболее компактный код (8720–9408 байт), что почти в 2 раза меньше GCC (стабильно 16992 байт) и ICX (16936–16984 байт). По времени выполнения все три компилятора показывают близкие результаты: GCC — 0.00084–0.00089 с, AOCC — 0.00086–0.00093 с, ICX — 0.00105–0.00109 с.
- **AES-128:** AOCC генерирует наиболее компактный код (12208–17784 байт), GCC — 16960–25520 байт, ICX — 16752–25432 байт. При -O2 ICX создаёт файл 24848 байт (+19% к GCC), что обусловлено агрессивной развёрткой циклов и multiversioning. По времени выполнения GCC с -Os показывает лучший результат (0.00120 с), ICX с -Os — 0.00126 с, AOCC с -O2 — 0.00131 с.

Результаты анализа ассемблерных листингов для программ aes.c¹ и optbench.c² свидетельствуют о применении компиляторами различных оптимизаций к участкам кода программы. Полный анализ сгенерированного ассемблерного кода для всех исследуемых функций, а также исходный код и результаты замеров представлены в репозитории³.

- **Функция Cipher:** все исследуемые компиляторы применяют встраивание функций, полностью устраняя накладные расходы на вызовы (call/ret) для функций AddRoundKey, SubBytes, ShiftRows внутри основного цикла.
- **Функции SubBytes и ShiftRows:** компиляторы AOCC и ICX реализуют слияние операций (operation fusion), объединяя этапы табличной подстановки и перестановки байтов в единый поток инструкций с полной раскруткой циклов для уменьшения обращений к памяти. У компилятора GCC прослеживается другой подход: он сохраняет исходный порядок следования функций, но выносит загрузку адреса таблицы sbox за пределы цикла и использует упаковку пар байт в 16-битные регистры с записью единой инструкцией movw для обеспечения компактности кода.
- **Функция AddRoundKey:** компилятор AOCC полностью раскручивает цикл в 16 последовательных инструкций xorb. Компилятор ICX сохраняет циклическую структуру, но добавляет проверку указателей: при отсутствии пересечения областей памяти массивов state и RoundKey используются 128-битные SIMD-инструкции (movdqu, pxor). Компилятор GCC оставил циклы.
- **Функция KeyExpansion:** компилятор GCC оптимизирует вычисление индексов, используя для операции взятия остатка $i \% 4$ быструю проверку битов testb \$3, а для умножения $i * 4$ использует addq \$4. Компилятор AOCC раскручивает первый цикл, но сохраняет второй с оптимизированной проверкой. Компилятор ICX выполняет полную раскрутку всех циклов, что устраняет накладные расходы на условные и безусловные переходы, но увеличивает размер кода.

¹Сравнительный анализ программы aes.c с описанием экспериментов: https://github.com/Quark1111/practice-compiler-optimization/blob/main/tables/aes/analysis_compilers.md (дата обращения: 17.09.2026).

²Сравнительный анализ программы optbench.c с описанием экспериментов: https://github.com/Quark1111/practice-compiler-optimization/blob/main/tables/optbench/analysis_compilers.md (дата обращения: 17.09.2026).

³Исходный код, результаты замеров и ассемблерные листинги размещены в репозитории: <https://github.com/Quark1111/practice-compiler-optimization> (дата обращения: 17.09.2026).

Итоговые выводы по оптимизации компиляторов

Анализ ассемблерных листингов выявил три принципиально различных подхода к оптимизации генерируемого кода:

- **GCC 13.3.0:** Приоритет отдается компактности кода и предсказуемости потока управления. Компилятор применяет классическое снижение мощности и инкремент указателей, избегая частых разветвок циклов для предотвращения разрастания бинарного файла.
- **АОСС 5.0.0:** Оптимизации направлены на параллелизм на уровне инструкций. Компилятор применяет разветвку циклов и слияние операций, что позволяет устранить накладные расходы на ветвления и обращения к памяти.
- **ICX 2026.0.0:** Ключевой особенностью является продвинутый анализ зависимостей по памяти. При отсутствии перекрытия областей памяти компилятор генерирует векторизованный код (SIMD), что повышает производительность, но приводит к увеличению размера исполняемого файла за счет генерации множественных путей выполнения.

6. Заключение

В ходе выполнения учебной практики были получены следующие результаты:

1. Выбран сценарий применения языка C ориентированный на задачи симметричного блочного шифрования и описана тестовая система на базе ОС Ubuntu 24.04.3 LTS с процессором AMD Ryzen 7 5700U (архитектура x86_64).
2. Обоснован выбор трёх компиляторов для исследования: GCC 13.3.0, АОСС 5.0.0 и ICX 2026.0.0.
3. Проведены замеры времени исполнения и размера исполняемых файлов для программ `optbench` и `AES-128` при трёх уровнях оптимизации (`-O0`, `-O2`, `-Os`): АОСС генерирует наиболее компактный код (12 208 байт для `AES` при `-Os`, что на 28% меньше GCC), GCC показывает лучшее время выполнения (0.00120 с при `-Os`).
4. Выполнен анализ влияния флага оптимизации `-O2` по сравнению с `-O0` для каждого компилятора: установлено, что GCC устраняет мёртвый код, сворачивает константы и заменяет тождественные операции на простые инструкции загрузки; АОСС дополнительно выполняет полную развёртку циклов (например, в `AddRoundKey` — 16 последовательных `xorb`), выносит инвариантные выражения за пределы циклов и удаляет избыточные присваивания; ICX полностью раскручивает циклы, заменяет деление на умножение и сдвиг, а также встраивает все вызываемые функции в тело `Cipher`, убирая кадр стека и обращения к памяти.
5. Проведён сравнительный анализ: компиляторы успешно применяют встраивание вспомогательных функций, полностью устраняя накладные расходы на вызовы. Однако по-разному обрабатывают циклы: в то время как GCC сохраняет исходную структуру, оптимизируя её через арифметику указателей, АОСС и ICX устраняют циклы путём их полной раскрутки и слиянием смежных операций.
6. Сформулированы выводы о применённых оптимизациях: GCC использует классические методы снижения вычислительной сложности и упаковки данных, сохраняя предсказуемость потока управления; АОСС фокусируется на параллелизме на уровне инструкций через массовую раскрутку циклов и слияние операций; ICX применяет глубокий анализ зависимостей по памяти для автоматической векторизации и замены дорогостоящих арифметических операций.

Список литературы

1. National Institute of Standards and Technology (NIST). *Advanced Encryption Standard (AES)*. FIPS PUB 197, 2001.
2. Kokke, P. *tiny-AES-c*. GitHub repository. URL: <https://github.com/kokke/tiny-AES-c> (дата обращения: 03.09.2026).
3. Ахо А., Лам М., Сети Р., Ульман Дж. *Компиляторы: принципы, технологии и инструменты*. 2-е изд. М.: Вильямс, 2008.
4. Free Software Foundation. *GNU Compiler Collection (GCC) Internals*. URL: <https://gcc.gnu.org/onlinedocs/gccint/> (дата обращения: 03.09.2026).
5. Advanced Micro Devices (AMD). *AMD Optimizing C/C++ Compiler (AOCC) User Guide*. URL: <https://www.amd.com/en/developer/aocc.html> (дата обращения: 03.09.2026).
6. Intel Corporation. *Intel® oneAPI DPC++/C++ Compiler Developer Guide and Reference*. URL: <https://www.intel.com/content/www/us/en/developer/tools/oneapi/dpc-compiler.html> (дата обращения: 03.09.2026).
7. Intel Corporation. *Intel® 64 and IA-32 Architectures Optimization Reference Manual*. URL: <https://software.intel.com/content/www/us/en/develop/download/intel-64-and-ia-32-architectures-optimization-reference-manual.html> (дата обращения: 03.09.2026).